Наследование

Подтемы занятия

- проблема дублирования кода
- определение наследования
- наследование в C++
- рекомендации по коду

Проблема дублирования кода

Сберегательный счёт

```
class SavingsAccount {
   string m_owner;
   double m_balance;
   deposit();
   // Невозможно снять денег больше,
   // чем есть на счёте.
   withdraw();
}
```

Расчётный счёт

```
class CheckingAccount {
   string m_owner;
   double m_balance;
   deposit();
   // Можно снять денег больше,
   // чем есть на счёте, в кредит.
   withdraw();
}
```

- поля *m_owner* и *m_balance* продублированы
- реализация метода *deposit* абсолютно одинаковая
- отличается только реализация метода *withdraw*

- наблюдается излишнее дублирование кода
- изменение общих методов потребует правок в 2 класса
- код не отражает общую природу банковских счётов

Решение проблемы - наследование

class Account

поля

- m_owner
- m_balance

методы

- deposit()
- withdraw()

class SavingsAccount

поля

- m_interest_rate

методы

- withdraw()

class CheckingAccount

поля

- m_overdraft_limit

методы

- withdraw()

Определение наследования

Наследование – это механизм ООП для создания нового класса на основе существующего. Наследник получает поля/методы родителя и может добавлять свои или изменять поведение.

Наследование стоит применять только тогда, когда выполняется правило ***Является***

X является разновидностью Y?

✅ **Да**

Сберегательный счёт является счётом, поэтому между ними можно сделать отношение наследования

❌ **Нет**

Двигатель не является автомобилем, поэтому между ними отношение наследования невозможно

Примеры наследования

Элементы интерфейса

- кнопки
- текстовые поля
- списки

Наследуются от

Widget

Объекты в видеоиграх

- игрок
- враг
- союзник

Наследуются от

GameObject

Исключения в C++

- потоки ввода файлов
- потоки ввода строк
- потоки ввода из консоли

Наследуются от

istream

Синтаксис наследования в C++

```
class Производный : public Базовый {                    // Класс Производный объявляется наследником от Базовый.
   Производный(параметры) : Базовый(параметры) {}    // В наследнике должен вызваться конструктор базового.
};
```

Инструменты наследования

- ***virtual*** используется для обозначения методов, которые планируется перегружать в наследниках
- ***override*** используется для переопределения методов базового класса, которые помечены *virtual*

Модификаторы доступа

public предоставляет доступ к полям внешнему коду

protected предоставляет доступ к полям внутри класса и его наследников

private предоставляет доступ к полям только внутри класса

Конец презентации

Разработчик

Марышев А.В.

Контакты для связи

Телеграм: @XekS_s